SYSTEM ARCHITECTURE & CONCURRENCY DESIGN

การออกแบบสถาปัตยกรรมระบบ Flash Sale และการจัดการ Concurrency

โจทย์ที่ 1 สถานการณ์ที่ 1

บริษัทกำลังจะจัดแคมเปญ "Flash Sale" แจกโค้ดส่วนลด 1,000 สิทธิ์ โดยจะเปิดให้กดพร้อมกันเวลา 12:00 น. คาดว่าจะมีผู้ใช้งาน (Traffic) เข้ามาพร้อมกันบนระบบทั้ง Web และ Mobile App ประมาณ 50,000 คนภายใน 5 นาทีแรก

1. ในฐานะ SA ให้ออกแบบ High-Level System Architecture สำหรับระบบนี้ คุณจะใช้ Tech Stack หรือ Component ใดบ้าง (เช่น Database, Cache, Message Queue ฯลฯ) เพื่อให้ระบบไม่ล่ม?
2. คุณจะออกแบบ Logic อย่างไรเพื่อแก้ปัญหา Concurrency (Race Condition) เพื่อป้องกันไม่ให้คนกดรับโค้ดส่วนลดซ้ำซ้อนหรือแจกโค้ดเกิน 1,000 สิทธิ์?

1. High-Level System Architecture และ Tech Stack

เพื่อรองรับปริมาณผู้ใช้งานพร้อมกันประมาณ 50,000 คนในช่วงเวลาเปิดแคมเปญและป้องกันระบบล่ม จึงเลือกใช้ Tech Stack และ Component ตามลำดับชั้นการทำงานดังต่อไปนี้

ลำดับ		Tech Stack	หน้าที่และการทำงานหลัก
1	**Edge & Traffic Control**	**Cloudflare**	แคชหน้า Static ไว้ที่ Edge 100% เพื่อไม่ให้เซิร์ฟเวอร์เสีย CPU ส่งหน้าเว็บ และตั้ง Rate Limit กัน Bot
2	**API Gateway**	**Nginx Ingress**	ทำ Per-User Rate Limiting โดยจำกัดให้ 1 User ID กดได้ไม่เกิน 1 ครั้งต่อวินาที และตรวจสอบ JWT Signature
3	**Application Service**	**Node.js**	ประมวลผลคำขอเนื่องจากรองรับ Concurrent Connections ได้สูงและใช้หน่วยความจำ (Memory) น้อย โดยสามารถตั้งค่า Pre-scale Pods รอไว้ล่วงหน้าได้
4	**In-Memory**	**Redis Cluster**	ใช้ตรวจสอบสิทธิ์และตัดโควตาจบใน RAM อย่างรวดเร็ว
5	**Message Broker**	**RabbitMQ**	ทำหน้าที่เป็น Shock Absorber รับข้อมูลเฉพาะ 1,000 คนที่ได้สิทธิ์ เพื่อส่งต่อไปบันทึกลงฐานข้อมูลตามจังหวะที่ฐานข้อมูลรองรับไหว
6	**Database**	**MySQL (InnoDB)**	เก็บข้อมูล Transaction การรับสิทธิ์โดยรับโหลดการทำงานจาก Worker

2. การออกแบบ Logic แก้ปัญหา Concurrency (Race Condition)

สำหรับการแก้ปัญหา Race Condition ในฝั่ง Node.js ร่วมกับ Redis ห้ามเขียนโค้ดแบบสั่ง `await redis.get()` มาตรวจสอบค่าในเงื่อนไข `if` แล้วค่อยสั่งหักลบแยกบรรทัด แต่ให้ออกแบบขั้นตอนการทำงานแบบ Atomic ดังนี้

1. **การล็อกไม่ให้ผู้ใช้งานเดิมกดรับสิทธิ์ซ้ำด้วย `redis.sadd()`:** แทนที่จะตรวจสอบก่อนว่าผู้ใช้งานเคยรับสิทธิ์หรือยัง ให้ยิงคำสั่ง `await redis.sadd()` เพื่อนำ `user_id` ของผู้ใช้งานใส่ลงใน Redis Set ทันที หากผู้ใช้งานรายนี้เพิ่งกดเป็นครั้งแรก Redis จะบันทึกค่าสำเร็จและตอบกลับมาเป็น `1` จึงอนุญาตให้ดำเนินการขั้นถัดไป แต่หากผู้ใช้งานเปิดหลายแท็บหรือเขียนสคริปต์ยิงคำขอรัวเข้ามาพร้อมกัน เธรดที่ตามมาจะได้ค่า `0` กลับไป ซึ่งระบบสามารถตอบกลับได้ทันทีว่า *"คุณรับสิทธิ์ไปแล้ว"*

2. **การตัดโควตา 1,000 สิทธิ์ ด้วย `redis.decr()`:** สั่ง `await redis.decr()` เพื่อหักลบเลขโควตาใน Redis ลงทีละ 1 สิทธิ์ คำสั่ง `DECR` จะเรียงคิวหักลบทีละรายการอย่างต่อเนื่อง หากหักลบแล้วได้ตัวเลขตั้งแต่ `999` ลงมาจนถึง `0` แสดงว่าผู้ใช้งานอยู่ในกลุ่ม 1,000 คนแรกที่ทันสิทธิ์ ให้ส่ง `user_id` เข้าสู่ Queue เพื่อนำไปบันทึกลง Database ต่อไป

3. **การจัดการเมื่อสิทธิ์เต็มและการเพิ่มประสิทธิภาพด้วย Lua Script:** หากผู้ใช้งานคนที่ 1,001 กดเข้ามา ตัวเลขที่ได้กลับมาจากคำสั่งหักลบจะกลายเป็น `-1` ซึ่งแสดงว่าสิทธิ์เต็มแล้ว ให้สั่ง Rollback คืนค่า และตั้งตัวแปร Flag ใน Memory ของ Node.js เป็น `isSoldOut = true` นอกจากนี้ยังสามารถใช้ไลบรารี `ioredis` รวมขั้นตอนการตรวจสอบรายชื่อและการหักโควตาไว้ใน `redis.defineCommand()` เพื่อเพิ่มประสิทธิภาพการทำงานของระบบให้ดียิ่งขึ้น